

TL1 Entorno de trabajo

Índice

1. [Introducción](#)
2. [numpy](#)
3. [scipy](#)
4. [matplotlib](#)
5. [scikit-learn](#)
6. [datasets](#)

Cita

Juan Ciscar, Alfons. "Material docente de la asignatura [Aprendizaje Automático \(APA\)](#)". [Grado de Inteligencia Artificial \(GIAR\)](#), [Escuela Técnica Superior de Ingeniería Informática \(ETSINF\)](#), [Dep. de Sistemas Informáticos y Computación \(DSIC\)](#), [Universitat Politècnica de València \(UPV\)](#), primer semestre del curso 2026/27.

1 Introducción

Entorno virtual: crea un entorno virtual para APA

```
In [ ]: %%sh -n
git clone git@github.com:AlfonsJ/2027_02_28_APA2627.git
cd 2027_02_28_APA2627
uv venv --seed --managed-python
source .venv/bin/activate
pip install numpy matplotlib pandas scipy scikit-learn kagglehub datasets torch torchvision torchaudio
pip install ipywidgets ipykernel
python -m ipykernel install --user --name APA2627
deactivate
```

numpy: librería de computación científica con ndarrays (vectores n-dimensionales)

scipy: librería de funciones científicas basada en numpy

matplotlib: librería para hacer gráficas basada en numpy

scikit-learn: librería de aprendizaje automático basada en numpy, scipy y matplotlib

datasets: librería Hugging Face (HF) para acceder y compartir datasets HF

Objetivo: familiarizarse con las librerías anteriores

2 numpy

numpy: librería de computación científica con ndarrays (vectores n-dimensionales)

Web numpy: <https://numpy.org/>

- **Documentation:** [absolute beginner's guide](#), [user guide](#), [reference guide](#) y [contributor's guide](#)
- **Learn:** [beginners](#), [advanced](#) y [talks](#)

User guide: <https://numpy.org/doc/stable/user/index.html#user>

- **Getting started:** recomendable si no has usado numpy anteriormente
- **Fundamentals and usage:** recomendable leer la sección sobre [broadcasting](#)

Ejercicio: traduce a numpy el siguiente pseudo-script numpy

```
In [ ]: %%script true
import numpy as np

X = matriz [-1, 1; 0, 2; 2, 0; 3, 3]

N = número de filas de X

mean = media de los datos

X_tilde = X - media

Sigma = matriz de covarianzas empírica calculada a partir de X_tilde
```

Solución:

```
In [ ]: import numpy as np
X = np.array([ [-1, -1], [0, 2], [2, 0], [3, 3] ]); X
```

```
Out[ ]: array([[ -1, -1],
               [  0,  2],
               [  2,  0],
               [  3,  3]])
```

```
In [ ]: N = len(X); print(N, X.shape[0])
4 4
```

```
In [ ]: mean = X.mean(axis=0); mean
```

```
Out[ ]: array([1., 1.])
```

```
In [ ]: X_tilde = X - mean; print(f'Broadcasting: {X.shape} - {mean.shape} -> {X_tilde.shape}')
X_tilde
```

```
Broadcasting: (4, 2) - (2,) -> (4, 2)
Out[ ]: array([[ -2., -2.],
               [ -1.,  1.],
               [  1., -1.],
               [  2.,  2.]])
```

```
In [ ]: Sigma = 1/N * X_tilde.T @ X_tilde; print(Sigma, '\n', np.cov(X, rowvar=False, bias=True))
[[2.5 1.5]
 [1.5 2.5]]
[[2.5 1.5]
 [1.5 2.5]]
```

3 scipy

scipy: librería de funciones científicas basada en numpy

Web scipy: <https://scipy.org/>

- **Documentation:** [user guide](#) y [reference guide](#)

scipy.stats: [distribuciones de probabilidad](#)

scipy.special: logit, softmax, lse, etc.

Ejercicio: traduce a scipy el siguiente pseudo-script scipy

```
In [ ]: %%script true
        from scipy import stats

        Y = array 1d con 10 muestras de una Bernoulli con probabilidad 0.25 de 1
        Y = array 2d con 3 muestras de una categórica para 3 clases con probabilidades (0.3, 0.2, 0.5)
        Y = array 1d con 5 muestras de una normal 1d estándar
        Y = rejilla 1d de 5 puntos en [-1, 1]
        pdf = array 1d con la pdf de una normal 1d estándar en Y
        Y = rejilla 2d de 3*2 puntos en [-1, 1]**2
        pdf = array 2d con la pdf de una normal 2d estándar en Y
```

Solución:

```
In [ ]: import numpy as np; np.random.seed(seed=23); from scipy import stats;
Y = stats.bernoulli(0.25).rvs(10); Y
```

```
Out[ ]: array([0, 1, 1, 0, 0, 0, 0, 0, 0, 0])
```

```
In [ ]: Y = stats.multinomial(1, [0.3, 0.2, 0.5]).rvs(3); Y
```

```
Out[ ]: array([[0, 1, 0],
               [1, 0, 0],
               [0, 0, 1]])
```

```
In [ ]: Y = stats.norm(0, 1).rvs(5); Y
```

```
Out[ ]: array([-0.16834479, -0.35671283,  0.85026465, -0.55916487, -0.85045193])
```

```
In [ ]: Y = np.linspace(-1, 1, 5); pdf = stats.norm(0, 1).pdf(Y); print(Y, pdf)
[-1.  -0.5  0.   0.5  1. ] [0.24197072 0.35206533 0.39894228 0.35206533 0.24197072]
```

```
In [ ]: m = np.zeros(2); S = np.eye(2); Y = stats.multivariate_normal(m, S).rvs(3); Y
```

```
Out[ ]: array([[ 0.27549799, -0.23764935],
               [-1.08664971,  0.87068849],
               [-0.36145286,  1.23030584]])
```

```
In [ ]: Y = np.linspace(-1, 1, 3); Y1, Y2 = np.meshgrid(Y, Y); Y = np.c_[np.ravel(Y1), np.ravel(Y2)]
pdf = stats.multivariate_normal(m, S).pdf(Y); print(Y, '\n', pdf)
```

```
[[ -1.  -1.]
 [  0.  -1.]
 [  1.  -1.]
 [ -1.   0.]
 [  0.   0.]
 [  1.   0.]
 [ -1.   1.]
 [  0.   1.]
 [  1.   1.]]
[0.05854983 0.09653235 0.05854983 0.09653235 0.15915494 0.09653235
 0.05854983 0.09653235 0.05854983]
```

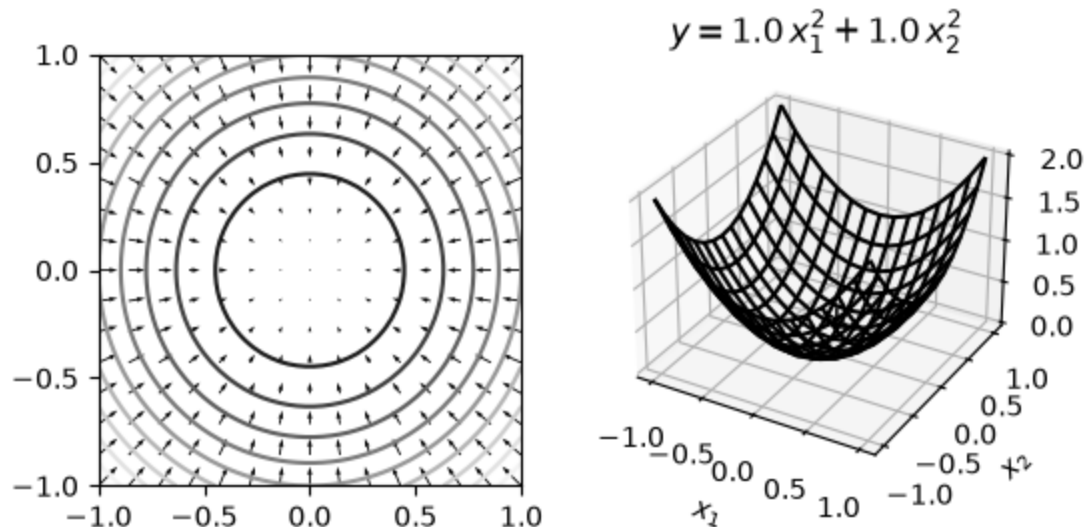
4 matplotlib

matplotlib: librería para hacer gráficas basada en numpy; web <https://matplotlib.org/>

- **Plot types:** [pairwise data](#), [gridded data](#), etc.
- **User guide:** [quick start guide](#) recomendable si no has usado matplotlib anteriormente
- **Otros:** [tutorials](#), [examples](#) y [API reference](#) para consultar cuando convenga

Ejercicio: estudia el código del siguiente script y prueba valores de a en $\{-1, 1\}^2$

```
In [ ]: import numpy as np; import matplotlib.pyplot as plt
R = np.linspace(-1, 1, 50); X1, X2 = np.meshgrid(R, R); X = np.c_[np.ravel(X1), np.ravel(X2)]
a = np.array([1, 1]); Y = (np.square(X) @ a).reshape(X1.shape)
fig = plt.figure(figsize=(6, 2.8)); ax = plt.subplot(121); ax.contour(X1, X2, Y, 10, cmap='Greys_r')
r = np.linspace(-1, 1, 15); x1, x2 = np.meshgrid(r, r); x = np.c_[np.ravel(x1), np.ravel(x2)]
neg_grad = -x @ np.diag(2*a); ax.quiver(x[:,0], x[:,1], neg_grad[:,0], neg_grad[:,1], color='black', angles='xy')
ax = plt.subplot(122, projection='3d'); ax.set_xlabel('$x_1$'); ax.set_ylabel('$x_2$')
ax.set_title(f'$y={a[0]:.1f}x_1^2 + {a[1]:.1f}x_2^2$'); ax.text(0, 0, 0, '*', fontsize=20)
ax.plot_wireframe(x1, x2, a[0]*x1**2 + a[1]*x2**2, color='black');
```



5 scikit-learn

scikit-learn: librería de aprendizaje automático basada en numpy, scipy y matplotlib

Web scikit-learn: <https://scikit-learn.org/stable/index.html>

- **User guide:** dividida en 15 partes
- **API:** para consultar cuando convenga
- **Examples:** para consultar cuando convenga
- **Community:** blog
- **Getting started:** recomendable si es la primera que usas scikit-learn
- **Glossary** de términos comunes y elementos API
- **FAQ:** interesante; ver [Will you add GPU support?](#)
- **Support:** listas de correo, [github discussions](#), [github bug tracker](#), etc.
- **Related projects:** algunos activos e interesantes

Github scikit-learn: <https://github.com/scikit-learn>

User guide: las siguientes partes son especialmente relevantes en APA

- **Supervised learning:** linear models, LDA/QDA, SGD, nearest neighbors, decision trees y ensembles
- **Unsupervised learning:** Gaussian mixture models, clustering y density estimation
- **Model selection and evaluation:** cross-validation, hyper-parameter tuning y metrics and scoring

Ejercicio: empieza con el [getting started](#) si es la primera que usas scikit-learn

6 datasets

datasets: librería Hugging Face (HF) para acceder y compartir datasets HF

Web datasets: get started, load a dataset from the hub, howtos, audio, vision, text, tabular, etc.

Ejercicio: comprueba que puedes leer MNIST, Fashion-MNIST y CIFAR-10

```
In [ ]: import numpy as np; import datasets
ds = datasets.load_dataset("ylecun/mnist").with_format("numpy"); ds
```

```
Out[ ]: DatasetDict({
  train: Dataset({
    features: ['image', 'label'],
    num_rows: 60000
  })
  test: Dataset({
    features: ['image', 'label'],
    num_rows: 10000
  })
})
```

```
In [ ]: ds = datasets.load_dataset("zalando-datasets/fashion_mnist").with_format("numpy"); ds['train']
```

```
Out[ ]: Dataset({
  features: ['image', 'label'],
  num_rows: 60000
})
```

```
In [ ]: ds = datasets.load_dataset("uoft-cs/cifar10").with_format("numpy"); ds['train']
```

```
Out[ ]: Dataset({
  features: ['img', 'label'],
  num_rows: 50000
})
```